RS-232 Instruction Set

Syntax

Commands consist of a two letter mnemonic and one or more parameters. Commands which end with a question mark (?) will return a value. Commands which end with an exclamation point (!) write the current parameter value to EEPROM for use after the next restart. Commands which end in an exclamation point and a question mark (!?) return the value stored in EEPROM.

All data is communicated in ASCII codes. Commands are case insensitive, and spaces (ASCII 32_{10}) are ignored. Commands are processed when a carriage return (ASCII13_{10}) is received. Returned values are delimited with commas (in the case of multiple returned values) or a carriage return (in the case of a single or the last returned value). Commands available to the end user are in **bold**: some commands are for factory use only and a special code must be transmitted to enable these commands. Parameter lists are enclosed in curly brackets { }, the brackets are not part of the command.

On reset, the unit will transmit the characters: PRS_10 with a carriage return.

Initialization

RS 1

Restart. This command will restart the PRS10's microcontroller just like power-on. (It is not necessary to send a **RS** command on power-up.) All values will return to the values stored in EEPROM. (verbose mode disabled, 10 MHz set to last stored value, etc.) The frequency lock-loop will be disabled until the microcontroller verifies that the unit is warmed-up and that a useful signal level is present. Example**: RS 1** will cause the unit to restart.

VB{0 or 1}
VB?

Set verbose mode. The verbose mode is useful when a human is communicating with the PRS10 using a terminal program (the PRS10 will provide an "OK", command prompts, etc.) The verbose mode should be disabled when a computer program is communicating with the PRS10 (where format characters would interfere.) **Examples: VB0** disables the verbose mode (this is the power-on default mode.) **VB1** will enable the verbose mode.

ID?

Identify. This command returns an identification string which includes the serial number and firmware version of the PRS10. **Example: ID?** will return the identification string PRS10_3.15_SN_12345 (model _firmware version_serial-number).

SN?
SN{value}
SN!
SN!?

Serial number. This command returns the unit's serial number. **Example: SN?** will return 21567 for a unit with serial number 21567. The command to write and burn a serial number are for factory use only.

ST?

Status. This command will return a six number string corresponding to the values of the six status bytes. Each number will range between 0 and 255, and will be separated by commas. (For definitions of the status bytes, refer to the end of the detailed command descriptions.)

LM{value} value = 0, 1, 2, or 3
LM?
LM!
LM!?

Lock mode pin configuration. This command is used to configure the LOCK/1PPS output (pin 1 on the main connector, J100.) The LOCK/1PPS pin may be configured per the following table:

LM	Description of LOCK/1pps Output
0	Output goes low when locked to Rb, pulses high for 10 µs at 1 Hz 1pps locking pre-filter disabled
1	Output goes low when locked to Rb, pulses high for 10 µs at 1 Hz 1pps locking pre-filter enabled (default)
2	Output goes low when locked to Rb, 1pps is disabled
3	Output goes high when locked to Rb, 1pps is disabled

The default value is 1, so that pin 1 will go low when the unit is locked to rubidium, and will pulse high for 10 µs at a 1 Hz rate. The position of the 1pps pulse may be moved with the PP command. **Example: LM?** Could return 1, indicating that the unit is in its default configuration so that the lock pin goes low when locked to Rb, pulsing high for 10 µs at a 1 Hz rate. To configure the unit for no 1pps output, the command string **LM 2** followed by **LM !** will change the unit's power on default for no 1pps output.

RC 1
RC!

Recall. This command is used to return all values in EEPROM to the values which were present when the unit was first shipped from the factory (except for the unit-start and lamp-start counters.) This command should be used if you have been writing values to EEPROM and have somehow corrupted the operation of the device. Executing this command may require calibration of the unit, as the frequency calibration values are also returned to their factory values. The unit will be restarted after the values in EEPROM have been restored to their factory values. **Example: RC 1** will return all calibration values to the values which were determined for the unit when it was manufactured and restart the unit. The RC! command is a factory-only command which writes all of the current parameter values to the EEPROM.

Frequency Lock-loop Parameters

LO{value} value = 0 or 1
LO?

Lock. This command can be used to stop the frequency lock-loop (FLL). It is essentially the same as setting the gain parameter to zero. It may be desirable in a particular application to stop the FLL and set the frequency control value for the 10MHz oscillator manually. (See the FC command.) **Example: LO 0** will stop the FLL. **LO?** will return a value of 0 (if the FLL is not active) or 1 (if the FLL is active.)

FC?
FC{high,low} $0 \leq \text{high} \leq 4095$ $1024 \leq \text{low} \leq 3072$
FC!
FC!?

Frequency control. These commands allow direct control of the 22bit value which controls the frequency of the 10 MHz ovenized oscillator. Normally, this value is controlled by the FLL control algorithm, however, the FLL may be stopped, and the value adjusted manually. (See the LO command.)

Two 12-bit DACs are scaled (by 1000:1) and summed to provide a varactor voltage which controls the frequency of the 10 MHz oscillator. The low DAC, which operates over half its range (to avoid FFL oscillations at the roll-over to the high DAC) provides a LSB frequency resolution of $1.5{:}10^{-12}$. The high DAC, which has a nominal value of 2048, has a LBS resolution of $1.5{:}10^{-9}$. These DACs provide a total tuning range of about ±3 ppm.

Example: Suppose a unit's FLL has been operating for some time and has settled. An **FC?** will return the current value of the DAC pair which might be 2021,1654. (Tracking the FC value over a long period of time tells you about the frequency variations of the 10MHz

crystal. The FC values will change to correct for variations in the crystal frequency due to aging and ambient conditions.)

Both DACs may be set to any value in the range specified above. **Example: FC 2048,2048** will set the 10MHz oscillator back to the middle of its tuning range. However, it is possible to set the frequency of the 10 MHz oscillator so far from the correct frequency that the FLL signal disappears, making the lock impossible. If this happens, the last saved FC value may be read from EEPROM with the **FC!?** command and restored with the **FC{high,low}** command.

The **FC!** command is used to save the current FC values in the unit's EEPROM. The **FC!?** Command may be used to read the value which is stored in the EEPROM. The value stored in EEPROM is used to set the 10 MHz at startup, before the FLL can be established. Occasionally while the unit is operating (at about 20 minutes after power-on and once a day there after) the program will write a new value to EEPROM to correct the value for crystal aging. **Example: FC!?** will return four values (separated by commas), the number of power cycles the unit has undergone, the number of times the FC pair has been written to EEPROM, and the value of the FC pair (high, low) which is used at turn-on and restart.

DS?

Detected signals. This command returns two numbers corresponding to the synchronously detected signals at the modulation frequency, ω_{mod}, and at twice the modulation frequency, $2\omega_{mod}$.

The first number, the amplitude of the signal at ω_{mod}, is the error signal in the rubidium frequency lock loop. The value is proportional to the instantaneous frequency error of the 10 MHz oscillator as detected by the physics package. The value may be large when the unit is first locking, and will bobble around zero in steady state. Each LSB corresponds to about 15 μVrms of signal at ω_{mod}.

The second number is the amplitude (in millivolts rms) of the synchronously detected signal at twice the modulation frequency, $2\omega_{mod}$. The amplitude of this signal is proportional to the strength of the rubidium hyperfine transition signal.

The returned value is a spot measurement taken over just one cycle of the modulation frequency. Since the signals have several Hz of equivalent noise bandwidth, they will be rather noisy.

Example: DS? could return 55,800 indicating a small error signal and a strong resonance signal.

SF{value} -2000 ≤ value ≤ +2000
SF?

Set frequency. This command is used to override the internal calibration pot (or external calibration voltage) to set the frequency directly, relative to the calibration values in EEPROM (see the SP and MO commands.) The command sets the frequency offset in units of parts in 10^{-12} (corresponding to a frequency resolution of 10 μHz at 10 MHz.) The SF? command will return the currently set relative frequency value (with a range of ±2000) whether the value comes from the internal calibration pot position, an external frequency control voltage, an SF command, or from the external 1pps phase lock loop control algorithm. However, SF set command is ignored if the unit is phase-locked to an external 1pps signal. (To re-establish direct control via the SF command, the PLL must be disabled. See PL 0 command.)

Example: SF 100 will set the frequency 100 x 10^{-12} (or 0.001 Hz) above the stored calibration value, and the **SF?** command will return 100.

Data from the SF command cannot be saved when the power is turned off. (To do this type of calibration, see the SP and MO commands.) Once executed, the SF command will disable the analog channels (internal calibration pot and external calibration voltage) until the power is cycled or the unit is restarted.

SS?
SS{value} 1000 ≤ value ≤ 1900
SS!
SS!?

Set slope. This command is used to read the slope calibration parameter for the SF command. This parameter compensates for a variety of factors which affect the magnitude of the coefficient between magnetic coil current and transition frequency. **Example: SS?** might return 1450, the nominal parameter value. This calibration parameter may not be altered by the end user.

The (factory only) SS! command is used to store the current value of the SS parameter to the unit's EEPROM. The SS!? will return the value of the SS parameter which is used on power-up or restart.

GA?
GA{value} 0≤ value ≤ 10
GA!
GA!?

Gain. This command sets the gain parameter in the frequency lock-loop algorithm. Higher gain values have shorter time constants, (the time constant is the time it takes for the frequency lock-loop to remove 67% of the frequency error) but have larger equivalent noise

bandwidths (which will reduce the short-term stability of the 10 MHz output.) A gain of 0 will stop the frequency lock-loop so that the frequency of the output is determined by the 10MHz ovenized oscillator alone. The gain setting, approximate time constants, and approximate equivalent noise bandwidths are detailed in the following table. The gain parameter is set automatically by the program, however, the user may want control over the parameter in special circumstances. **Example: GA7** will set the gain parameter to 7, which has a time constant of about 2 s, which is a typical value for normal operation. **GA?** could return a value of 8 just after restart, which has a short time constant of about 1 s to assist the initial frequency locking. Setting the gain parameter during the first 6 minutes after turn-on or restart will abort the automatic gain sequencing.

Command	Time Constant (seconds)	Noise Bandwidth (Hz)
GA 0	Infinite	0
GA 1	128	0.002
GA 2	64	0.004
GA 3	32	0.008
GA 4	16	0.016
GA 5	8	0.032
GA 6	4	0.064
GA 7	2	0.128
GA 8	1	0.256
GA 9	0.5	0.512
GA 10	0.25	1.024

The **GA!** command stores the current value of the frequency lock loop gain parameter into the unit's EEPROM. **Example:** If the current value of the gain is 6, the command **GA!** will write 6 to the unit's EEPROM which will be used to initialize the gain parameter after the next power-on or restart. Then **GA!?** will return a 6.

PH?
PH{value} 0≤ value ≤ 31
PH!
PH!?

Phase. This command is used to set the phase of the synchronous detection algorithm. The frequency lock-loop (FLL) uses the in-phase component of the photo-signal at the modulation frequency (70 Hz) as the error signal for the FLL. The phase between modulation source and the error signal is affected by phase shifts in the modulation and signal filters and by optical pumping time constants. This parameter corrects for the accumulation of all of these phase shifts. Each modulation cycle consists of 32 phase slots, so each phase increment corresponds to 11.25°. **Example: PH?** would typically return a value of 24.

The PH! command is used to write the current phase parameter into the unit's EEPROM. This is a factory only command. The value which is burned in EEPROM is used on power-on and restart, and may be queried by the **PH!?** command. **Example: PH!?** could return a typical value of 24.

Frequency Synthesizer Control

A frequency synthesizer, which uses the 10 MHz OCXO as a frequency reference, is used to generate the RF which sweeps the rubidium hyperfine transition. The frequency synthesizer multiplies the 10 MHz by a factor M = 19 * (64*N + A) / R, to generate a frequency near 6.834 GHz. (The factor of 19 is from frequency multiplication in the step recovery diode, and the other terms come from the operation of the dual modulus frequency synthesizer integrated circuit.)

The apparent transition frequency is different for each physics package, due mostly to variations in the fill pressure of the resonance cell. The frequency synthesizer parameters, R, N and A, are used to adjust the frequency synthesizer's output frequency to the closest frequency just above the apparent transition frequency, then the magnetic field is set to move the transition frequency up to the synthesizer frequency.

During frequency locking, the frequency of the 10 MHz OCXO is adjusted to maintain the output of the frequency synthesizer on the rubidium hyperfine transition frequency. Initial calibration of the unit will involve finding the synthesizer parameters and magnetic field value which will lock the 10 MHz OCXO at exactly 10 MHz.

During the lifetime of the unit, there will be some aging of the physics package, which will cause the apparent transition frequency to change. This is usually corrected by minor calibration adjustments of the magnetic field strength, which provides a setting resolution of a few parts in 10^{-12}. (See the MO command.) However, if the magnetic field strength reaches its lower or upper limit, it is necessary to change the frequency synthesizer parameters, which can change the output frequency in steps of about one part in 10^{-9}.

The table in Appendix A details the values for R, N and A for the range of frequencies needed.

SP?
SP {R,N,A} $1500 \leq R \leq 8191$ $800 \leq N \leq 4095$ $0 \leq A \leq 63$
SP!
SP!?

Set Parameters. This command is used to set or query the frequency synthesizer's parameters, which will coarsely adjust the unit's output frequency. These parameters may need to be adjusted if the unit cannot be calibrated by magnetic field adjustment.

Example: During calibration, a unit's 10 MHz output frequency is found to be low by 0.010 Hz, and the magnetic field offset adjustment is already at its maximum. (See the MO command.) Sending the **SP?** command returns the current values of R, N and A which are 2610,1466,63 in this example. This corresponds to line 38 in the table in Appendix A. To *increase* the frequency of the 10 MHz output, we select the next *higher* setting, line 37, which will *increase* the frequency by 0.01986 Hz. To do this, we send the command **SP 5363,3014,22** (which are the parameters from line 37). Waiting for the frequency to settle, we now measure the output to be about 0.0098 Hz high. Now the magnetic field is adjusted down to calibrate the unit to exactly 10 MHz. (The **SP!** command is used to save these new values in EEPROM for the next power-on or restart. Also see the MO command for adjusting the magnetic field.)

The **SP!** command is used to write the current frequency synthesizer parameters to the unit's EEPROM for use after the nest restart or power-on cycle. This command is used after the SP command is used during the calibration of the unit. **Example: SP!** will write the frequency synthesizer parameters (R, N and A) which are currently in use to the unit's EEPROM. **SP!?** will return the values for R, N and A which are currently in the unit's EEPROM. The **SP!?** command may be used to verify that the **SP!** write command executed correctly.

Magnetic field Control

A magnetic field coil inside the resonance cell is used to tune the apparent hyperfine transition frequency. The magnetic field is controlled by a 12-bit DAC. Increasing the magnetic field will increase the hyperfine transition frequency, which will increase the frequency of the 10 MHz output. The transition frequency may be tuned over about $\pm 3 \times 10^{-9}$ by the magnetic field, which corresponds to ±0.030 Hz at 10 MHz. The output frequency (at 10 MHz) tunes quadratically with field strength, and $\Delta f(\text{Hz}) \approx 0.08 * (\text{DAC}/4096)^2$.

A minimum magnetic field should always be present to avoid locking to the wrong Zeeman component of the hyperfine transition, so the 12-bit DAC may be set from 1000 to 4095 with 3000 being the nominal midscale value. (A DAC value of 1000 corresponds to about 6% of the full-scale frequency tuning range, 3000 corresponds to about 53%, while 4095 is 100% of the full-scale range.)

To help cancel frequency shifts due to external magnetic fields, the current in the coil is switched at a 5 Hz rate. The frequency lock-loop averages over a full period of the switch rate to avoid injecting a spur at 5 Hz onto the 10 MHz control signal. The switching of the magnetic field is enabled at power-on and restart, but may be turned on or off by RS-232 command. (see MS command.)

The commands associated with magnetic field control (MO, MS, and MR) allow direct control of the magnetic field circuitry. Most users will not want to control the magnetic field directly, but will instead allow the program to read the frequency calibration pot or external control voltage and then control the magnetic field. If they want software control of the unit's

calibration, they may choose to use the SF commands, which disable the analog control and allow the frequency to be adjusted over a range of $\pm 2000 \times 10^{-12}$. (The program will linearize the magnetic field control of the frequency offset with either analog or software control.)

MS?
MS{0 or 1}

Magnetic switching. The MS command is used to turn off or on the 5Hz switching of the frequency tuning magnetic field. Magnetic switching is enabled when the unit is powered-on or after a restart. (Since the PRS10 is calibrated with the field switching enabled, turning off the field switching may alter the calibration.) **Example: MS 1** will turn on the magnetic field switching, and **MS 0** will turn it off. **MS?** will return a "1" if the field switching is currently enabled.

MO?
MO{value} $2300 \leq \text{value} \leq 3600$
MO!
MO!?

Magnetic offset. The magnetic offset is the value, determined when the unit is calibrated, which calibrates the unit to 10 MHz. The restricted range is necessary to allow room for user calibration via the internal frequency calibration pot or by an external voltage. If the unit cannot be calibrated to 10 MHz within the allowed range of MO values, then a different setting for the frequency synthesizer is required. (See SP command and the table in Appendix A. **Example: MO 3000** sets the magnetic offset to 3000, which is its nominal (mid-linear scale) value. The **MO?** command reads back the current value of the magnetic offset. **MO!** is used to store the current value of the magnetic offset parameter to EEPROM for use after the next restart. **MO!?** may be used to query the value stored in EEPROM. This value is used on power-up or restarts.

MR?

Magnetic read. This command returns the value that the 12-bit DAC is using to control the magnetic field. This value is computed from the magnetic offset value (see MO command) and the position of the internal frequency calibration pot, external calibration voltage, or value sent by the SF command.

The value is computed from the equation $DAC = \sqrt{(SF*SLOPE + MO^2)}$ where SF is the desired frequency offset in parts per 10^{-12} (from the cal pot position, the SF command, or the 1pps PLL and is in the range $-2000 < SF < 2000$), SLOPE is the SF calibration factor with a nominal value of 1450 (see SS command), and MO is the magnetic offset value. The returned value should be in the range of 1000 to 4095.

Example: MR? would return a value of 3450 if the magnetic offset is at 3000, the SF command requested an offset of +2000 x 10^{-12}, and the SS CAL factor has the nominal value of 1450.

Frequency Control

The frequency of the 10 MHz output may be adjusted in a number of ways: the internal calibration potentiometer may be set (accessible via a hole in the bottom plate), an external voltage (0 to +5.00 Vdc, applied to the interface connector pin 2) can override the internal pot, or, these analog channels may be overridden with a software command which sets the frequency directly.

When the unit turns on, or after a restart command, the control program will default to reading the analog channel for frequency calibration. (This is important to maintain compatibility with existing sockets.) The calibration pot and the external voltage control provide a full-scale tuning range of ± 2000 x 10^{-12}, with a worst case resolution of 5 x 10^{-12}.

All of the channels for calibrating the unit are linearized, so that the frequency characteristic will be linear with applied voltage, pot setting, or SF value even though the transition frequency changes quadratically with field strength.

One pulse per second (1pps) control

To facilitate system integration, the PRS10 provides a 1pps output which may be set over an interval from 0 to 999,999,999 ns with 1ns resolution. The unit also has the ability to measure the arrival time of a 1pps input over the same interval and with the same resolution.

The ability to time-tag a 1pps input allows the PRS10 to be phase-locked to other clock sources (such as the 1pps output from a GPS receiver) with very long time-constants. This is a very useful feature for network synchronization, and allows the configuration of a reliable Stratum I source at a very low cost.

TT?

Time-tag. This command returns the value of the most recent time-tag result in units of nanoseconds. If a new time-tag value is not available then -1 (the only case for which the returned value is negative) will be returned. **Example: TT?** would return the value 123456789 to indicate that the most recent 1pps input arrived 123,456,789ns after the 1pps output. Returned values range from 0 to 999999999.

TS?
TS{value} 7000 ≤ value ≤ 25000
TS!
TS!?

Time slope. This command is used to calibrate the analog portion of the time-tagging circuit. The analog portion is used to digitize the time of arrival with 1 ns resolution and 400 ns full-scale. (Counters are used for the portion of a time interval longer than 400 ns.) The analog circuit stretches the time interval between the 1pps input and the next edge of a internal 2.5 MHz clock by a factor of about 2000, and measures the duration of the stretched pulse by counting a 2.5 MHz clock. The analog portion of the time-tag result is calculated from the equation $\Delta T(ns) = counts * TS / 2^{16}$, where TS is the time slope value, which has a nominal value of 13,107.

Example: TS? might return 14,158 which is a time slope parameter value a bit above the nominal value, which would be required if the analog portion of the time-tagging circuit stretched the pulse by a bit less than a factor of 2000. **TS?** will return the current value of the time slope.

The TS! command is used to write the current value of the time slope parameter into the unit's EEPROM. The TS {value} and TS! are factory only commands. Example: TS! will write the current value of the time slope (which may be queried with the TS? command) to the unit's EEPROM. **TS!?** will return the time slope calibration factor which is in the unit's EEPROM.

TO?
TO{value} -32767 ≤ value ≤ 32768
TO!
TO!?

Time offset. This calibration value (in ns) is added to the measured time-tag value to reference the result to the 1pps output. To calibrate, the 1pps output is connected to the 1pps input and the time-tag is read with the TT? command. The returned value is subtracted from the current TO value and sent with the TO command to calibrate the offset.

Example: Suppose: the 1pps output is connected to the 1pps input. A time tag value, read with the TT? query, returns a value of 25ns. The TO parameter, read via the TO? query, returns a value of -1750ns. The command TO -1775 is sent to correct for the offset. After waiting about one second (to allow another time-tag value to be acquired) the next TT? query returns a value of 2ns (indicating a measurement of 2ns after the 1pps output) Waiting another second, the next TT? query returns the value 999,999,999ns (indicating 1 ns before the 1pps output). These values are consistent with a well calibrated time-tag offset.

Following calibration of the TO parameter, the **TO!** command is used to write the current value of the time offset to the unit's EEPROM. Example: **TO!** will write the current time-tag

offset value to the unit's EEPROM for use after the next power-up cycle or restart command. **TO!?** will return the value which is burned in the unit's EEPROM. (Note: Firmware revisions prior to Rev 3.23 do not allow user TO! commands. Check the firmware revision with the ID? command.

PP{value} 0 ≤ value ≤ 999999999

Place pulse. This command is used to move the 1pps output from its current position. The 1pps output can be moved earlier in time by 1 ns to 999999999 ns. Since the 1pps input time-tag is referenced to the 1pps output, changing the 1pps output placement will change the report time-tag values as well. (See the TT and TO commands.) **Example: PP 123456789** will move the 1pps pulse train earlier by 123,456,789 ns.

PS?
PS{value} 100 ≤ value ≤ 255
PS!
PS!?

Pulse slope calibration. This command is used to calibrate the analog portion of the 1pps output time delay circuit. This circuit is used to delay the 1pps pulse train with 1 ns resolution and 100 ns full-scale. (Counting logic is used for the portion of the time interval longer than 100 ns.) The pulse slope value corresponds to the DAC8 value which provides a delay closest to (but not exceeding) 100 ns. Example: PS 200 set the pulse slope to its nominal value of 200. (PS{value} is a factory only command.) The **PS?** command will return the current value of the pulse slope.

The PS! command writes the current value of the pulse slope to the unit's EEPROM for use after the next power-on or restart. This command is used after the pulse output analog output is calibrated. Example: PS! will write the current value of the pulse slope (which calibrates the 100 ns analog delay portion of the 1pps pulse delay circuit) to the unit's EEPROM. Note that PS! is a factory-only command.

1PPS Locking Control

To facilitate integration into systems which require very low aging, automatic calibration, or a traceable frequency standard, the PRS10 may be locked to an external 1pps input.

A second-order digital PLL is used to lock the unit's frequency (both the 10 MHz and 1pps outputs) to an external 1pps input with time constants ranging from 256 s to 65536s (about 4 minutes to about 18 hours).

When provided with an accurate and stable 1pps source, the unit will automatically align its 1pps output to the 1pps input and then adjust the frequency of the rubidium reference to

maintain the alignment over time. A typical application would lock the PRS10 to the 1pps output from a GPS receiver with a time constant of several hours.

Several commands and one status byte may be used to control and monitor the PLL, however, default values will allow units to lock to clean 1pps inputs without any software interaction.

PL?
PL{0 or 1}
PL!
PL!?

Phase lock control. This command may be used to disable the 1pps PLL, or to re-enable (and so restart) the 1pps PLL. The unit is shipped with the phase lock control enabled. This command would be used if the 1pps time-tagging were being used to measure the position of 1pps inputs and phase locking is not desired. **Example: PL 0** will disable the PLL to the 1pps inputs so that the frequency of the rubidium standard will not be affected by the 1pps inputs. **PL?** will return a "1" if the PLL to the 1pps is enabled. **PL!** is used to write the current value (0 or 1) to the EEPROM for use after the next start up. **PL!?** is used to query the value of the phase lock control parameter which is stored in the unit's EEPROM.

PT?
PT{value} $0 \leq \text{value} \leq 14$; $\tau_1 = 2^{(\text{value}+8)}$ seconds (256, 512, ... 4,194,304)
PT!
PT!?

Phase-lock integrator time constant. This command is used to set the PLL's integrator's time constant, τ_1, which phase-locks the PRS10 to an external 1pps input. The integrator time constant is equal to $2^{(\text{value}+8)}$ seconds. The default value is 8, which provides an integrator time constant of $2^{(8+8)}$ or 65536 seconds. Integrator's time constants can range from 256 to 4,194,304 seconds, or from about 4 minutes to 18 days. It is important to note that the natural time constant, τ_n, is different from the integrator time constant, as shown in the table below. The natural time constant is the best measure of the loop response. The PLL natural time constant spans between 8 minutes and 18 hours for PT values between 0 and 14.

Example: PT10 sets the integrator time constant to $2^{(10+8)}$ seconds, or about 72 hours. (Refer to Table below.) For PT10 the natural time constant is about 4.5 hours. **PT?** will return the current value of the time constant parameter. A phase lock time constant may be stored in EEPROM as a new default with the **PT!** command. The **PT!?** command may be used to verify the value stored in EEPROM.

The following case will illustrate the operation of the PLL: Suppose that the PRS10 has been phase locked to a stable 1pps reference for a very long time (several periods of τ_n) when the 1pps reference input makes an abrupt shift of +100ns (moving later in time). The PRS10's 1pps PLL algorithm will reduce the PRS10's frequency of operation (by adjusting its SF

parameter) to eliminate the 100ns phase shift between the 1pps reference input and the 1pps output. After the phase shift is eliminated, the PRS10 will settle to the "correct" operating frequency.

The PLL algorithm computes integral and proportional terms from time-tag measurements, adjusting the SF parameter to phase lock the 1pps output to the 1pps input. The table below shows the integral and proportional gain terms. For the nominal PT value of 8, the integral term is -0.055 SF bits per hour per ns of time-tag and the proportional gain is -0.25 SF bits per ns of time-tag.

Per the table below for PT8, if the input reference shifts by +100ns, the proportional term will adjust the SF by -0.25bits/ns * 100ns = -25 bits. Each SF bit corresponds to $1{:}10^{-12}$ of the operating frequency, and so the PRS10 frequency will be shifted by about -25×10^{-12}. The integral term will begin ramping by (-0.055bits/hour/ns) * 100ns, or by -5.5 bits per hour. The phase shift between the 1pps input and 1pps output will be gradually eliminated.

(Phase jumps of 100ns are quite common on 1pps outputs from GPS receivers, which are a likely 1pps reference to the PRS10. The corresponding frequency jumps of 25×10^{-12} may be excessive in some applications, and so a digital pre-filter is used to smooth the time-tag values before they are used by the PLL algorithm. See LM command.)

PLL Table for all PT values, assuming a stability factor, ζ=1.

PT Parameter Parameter set by PT command	Integrator Time-Constant (hours)	Integral Gain (SF bits per hour per ns of time-tag)	Proportional Gain (SF bits per ns of time-tag)	Natural Time-Constant Characterizes PLL response (hours)
0	0.07	-14.063	-3.95	0.14
1	0.14	-7.031	-2.80	0.20
2	0.28	-3.516	-1.98	0.28
3	0.57	-1.758	-1.40	0.40
4	1.14	-0.879	-0.99	0.56
5	2.28	-0.439	-0.70	0.80
6	4.55	-0.220	-0.49	1.12
7	9.10	-0.110	-0.35	1.59
8	18.20	-0.055	-0.25	2.25
9	36.41	-0.027	-0.17	3.18
10	72.82	-0.014	-0.12	4.50
11	145.64	-0.007	-0.09	6.36
12	291.27	-0.003	-0.06	8.99
13	582.54	-0.002	-0.04	12.72
14	1,165.08	-0.001	-0.03	17.99

PF?
PF{value} $0 \leq$ value ≤ 4 ; (value:ζ) : (0:1/4, 1:1/2, 2:1, 3:2, or 4:4)
PF!
PF!?

Phase-lock stability factor. This command is used to set the stability factor, ζ, of the 1pps PLL. The stability factor is equal to $2^{(value-2)}$. The default value is 2, which provides a stability factor of $2^{(2-2)} = 2^0 = 1$. Stability factors can range from 0.25 to 4.0. **Example: PF 1** sets the stability factor to 0.5, which will reduce the equivalent noise bandwidth of the PLL at the cost of increasing the ringing near the natural frequency (relative to the default settings). **PF?** will return the current value of the stability factor parameter. **PF!** may be used to write the current stability factor to the EEPROM for use as the new default. **PF!?** may be used to read the value of the stability factor which is stored in EEPROM.

PI?
PI {value} $-2000 \leq$ value ≤ 2000

Phase-lock integrator. This command is used to set the value of the integral term in the PLL's digital filter. It is not necessary to set this value, as it will be initialized by the PLL routine to the current frequency setting parameter when the PLL begins. Users may want access to the value to alter the PLL characteristics, or to investigate its operation. **Example: PI 0** will set the integrator in the PLL's digital filter to 0, which is the center of the ±2000 bit range. **PI?** will return the current value of the PLL integrator. (There are two terms which control the phase locking of the PRS10 to an external 1pps source: the integral term and the proportional term. The proportional term is equal to the value returned by an **SF?** minus the value returned by the **PI?**.)

Analog Control

SD{port}?
SD{port,value} $0 \leq$ port ≤ 7 and $0 \leq$ value ≤ 255 (factory only)
SD{port}!
SD{port}!?

Set DAC. This command is used to set (or read the settings of) an octal 8bit DAC which provides analog signals to control systems parameters. The command which sets values is only available to the factory. The command to query values may be used by all. The query command returns a single integer in the range of 0 to 255.

Port	Function
0	Controls the amplitude of the RF to multiplier in resonance cell
1	Controls the analog portion (0 to 99 ns) of the delay for the 1pps output

2	Controls the drain voltage for the discharge lamp's FET oscillator
3	Controls the temperature of the discharge lamp
4	Controls the temperature of the 10 MHz SC-cut crystal
5	Controls the temperature of the resonance cell
6	Controls the amplitude of the 10 MHz oscillator
7	Controls the peak deviation for the RF phase modulation

Example: SD2? could return the value 255 indicating that the unit has set the discharge lamp's FET drain voltage to the maximum (which it does while it is trying to start the lamp.)

The SD{port}! is a factory only command which writes the data from the corresponding SD port to the unit's EEPROM for use on subsequent restarts. **Example: SD3!?** will return the start-up value for SD3 (lamp temperature control value) which is stored in the unit's EEPROM.

Analog Test Voltages

AD{port}? port = 0,1,2,...15
Analog to digital. This command reads the voltage at the corresponding 12-bit ADC port and returns the voltage as a floating point number. Values can range from 0.000 to 4.998. The voltages correspond to various test points in the system per the following table. Note that this command can only query. **Examples: AD10?** could return the value 0.710 indicating that the case temperature sensor is at 71 °C (this sensor indicates a temperature which is about midway between the baseplate temperature and the lamp temperature.)

Command	Returned voltage
AD 0?	Spare (J204)
AD 1?	+24V(heater supply) divided by 10.
AD 2?	+24V(electronics supply) divided by 10
AD 3?	Drain voltage to lamp FET divided by 10
AD 4?	Gate voltage to lamp FET divided by 10
AD 5?	Crystal heater control voltage
AD 6?	Resonance cell heater control voltage
AD 7?	Discharge lamp heater control voltage
AD 8?	Amplified ac photosignal
AD 9?	Photocell's I/V converter voltage divided by 4
AD 10?	Case temperature (10 mV/°C)
AD 11?	Crystal thermistors
AD 12?	Cell thermistors
AD 13?	Lamp thermistors
AD 14?	Frequency calibration pot / external calibration voltage
AD 15?	Analog ground

AD{port}? 16 ≤ port ≤ 19

A/D via CPU's E-port. This command returns a value corresponding to the voltage present at the input to the microcontroller's octal 8bit ADC (port E on the MC68HC11). Only the first four ports are in use. The voltage corresponds to various test point in the system per the following table. **Example: AD17?** could return a value of 4.81 indicating that the 360 MHz RF synthesizer has acquired lock.

Command	Returned voltage
AD 16?	Varactor voltage for 22.48 MHz VCXO (inside RF synthesizer) / 4
AD 17?	Varactor voltage for 360 MHz VCO (output of RF synthesizer) / 4
AD 18?	Gain control voltage for amplifier which drives frequency multiplier / 4
AD 19?	RF synthesizer's lock indicator voltage (nominally 4.8 V when locked)

Status Bytes

ST?

Status query. This command returns the six system status bytes which are used to indicate the health and status of the unit. The values ranges from 0 to 255. The six status bytes are detailed in the tables below. A status bit will remained set until it is read, even though the condition which caused the error has been removed. Some status bits are not errors: for example, during warmup the status bytes may indicate that the lamp is not lit, temperatures are low, and the unit is not locked.

Example: Immediately after power is applied to a unit, the command **ST?** returns 16, 3, 21, 1, 2, 129. From the status byte definitions below, we see that the following conditions exist:

16… the lamp has not yet started
3… the RF VCXO has not yet locked
21… the lamp, crystal, and cells are all below their set point temperatures
1… the frequency lock has not been established
2… fewer than 256 1pps inputs have been qualified
129.. both the lamp and unit have been restarted

ST1 : Power supplies and Discharge Lamp

ST1 bit	Condition which sets bit	Corrective Action
0	+24 for electronic < +22 Vdc	Increase supply voltage
1	+24 for electronics > +30 Vdc	Decrease supply voltage
2	+24 for heaters <+22 Vdc	Increase supply voltage

3	+24 for heaters > +30 Vdc	Decrease supply voltage
4	Lamp light level too low	Wait: check SD2 setting
5	Lamp light level too high	Check SD2 setting
6	Gate voltage too low	Wait: check SD2 setting
7	Gate voltage too high	Check SD2 setting

ST2: RF Synthesizer

ST2 bit	Condition which sets bit	Corrective Action
0	RF synthesizer PLL unlocked	Query SP? verify values
1	RF crystal varactor too low	Query SP? verify values
2	RF crystal varactor too high	Query SP? verify values
3	RF VCO control too low	Query SP? verify values
4	RF VCO control too high	Query SP? verify values
5	RF AGC control too low	Check SD0? values
6	RF AGC control too high	Check SD0? values
7	Bad PLL parameter	Query SP? verify values

ST3: Temperature Controllers

ST3 bit	Condition which sets bit	Corrective Action
0	Lamp temp below set point	Wait for warm-up
1	Lamp temp above set point	Check SD3, ambient
2	Crystal temp below set point	Wait for warm-up
3	Crystal temp above set point	Check SD4, ambient
4	Cell temp below set point	Wait for warm-up
5	Cell temp above set point	Check SD5, ambient
6	Case temperature too low	Wait for warm-up
7	Case temperature too high	Reduce ambient

ST4: Frequency Lock-Loop Control

ST4 bit	Condition which sets bit	Corrective Action
0	Frequency lock control is off	Wait for warm-up
1	Frequency lock is disabled	Enable w/LO1 command
2	10 MHz EFC is too high	SD4,SP,10MHz cal,T_{amb}
3	10 MHz EFC is too low	SP, 10 MHz cal

4	Analog cal voltage > 4.9 V	Int cal. pot, ext cal. volt
5	Analog cal voltage < 0.1	Int cal. pot, ext cal. volt
6		
7		

ST5: Frequency Lock to External 1pps

ST5 bit	Condition which sets bit	Corrective Action
0	PLL disabled	Send PL 1 to enable
1	< 256 good 1pps inputs	Provide stable 1pps inputs
2	PLL active	
3	> 256 bad 1pps inputs	Provide stable 1pps inputs
4	Excessive time interval	Provide accurate 1pps
5	PLL restarted	Provide stable 1pps inputs
6	f control saturated	Wait, check 1pps inputs
7	No 1pps input	Provide 1pps input

ST6: System Level Events

ST6 bit	Condition which sets bit
0	Lamp restart
1	Watchdog time-out and reset
2	Bad interrupt vector
3	EEPROM write failure
4	EEPROM data corruption
5	Bad command syntax
6	Bad command parameter
7	Unit has been reset